produces:

```
I'm the child, the parent said to me 'Get a job!'
I'm the parent, the child said to me 'OK'
```

Let's walk this one through. The original, soon to be parent, interpreter calls IO.popen with the minus sign argument. At this point we now have two Ruby interpreters each of which moves forward from this point. The original interpreter gets an IO pipe back as new_pipe, and the new, child, interpreter gets nil. At this point, the parent can send text to the child using new_pipe.puts and can listen for text from the child using new_pipe.gets. From the child's perspective, new_pipe is nil, but it can communicate with the parent using the regular Kernel methods, gets to listen for input and puts to send output to the parent.

So, the if new_pipe expression – for the child, the pipe is nil and this expression is false, and the child goes down the else branch. For the parent, the pipe exists, the expression is true, and the parent goes down the main branch.

In the parent branch, the parent immediately uses new_pipe.puts to send a string to the child branch, then calls $stderr.puts to write something to the global standard error output. That string contains new_pipe.gets, meaning that it will block waiting for something to be sent from the child.

In the child branch, similar things happen. The $stderr.puts call includes a call to gets which is listening for the text coming from the parent process, and then the child puts text to be read by the parent process.

We're using the standard error port here rather than standard out, because standard error automatically flushes its text after being called. If we used standard output, we'd likely get the first part of each output statement interleaved while it waits for the text coming from the other process.

In addition to the popen method, some platforms support the methods fork, exec, and pipe. The file naming convention of many IO methods and open will also spawn subprocesses if you put a pipe character, |, as the first character of the filename. Note that you *cannot* create pipes using File.new; that method is just for files.

Independent Children

Sometimes we don't need to be quite so hands-on; we'd like to give the subprocess its assignment and then go on about our business. Later, we'll check to see whether it has finished. For instance, we may want to kick off a long-running external sort:

```
pid = spawn("sort testfile > output.txt")
 # The sort is now running in a child process
 # carry on processing in the main program

 # ... dum di dum ...

 # then wait for the sort to finish
Process.wait(pid)
```

The call to spawn here executes a system-level command and returns its process id. But it does not wait for the command to finish, so Ruby processing continues apace. Later, we issue a Process.wait call with the process id, which causes the parent process to wait for the child process running the sort to complete, and returns the child process ID.